

Automatic Image Analysis Script

June 30, 2026

Contents

1	Introduction to Automatic Image Analysis	2
1.1	Lecture Aim and the Inverse Problem	2
1.2	From Pixels to Interpretation	3
1.3	Data Types and Interpretation Tasks	5
1.4	Models, Data, and Evaluation	6
1.5	Topic Roadmap	8
2	Neural Networks and Backpropagation	8
2.1	From Perceptrons to Multilayer Networks	9
2.2	Losses, Optimization, and the Need for Gradients	9
2.3	Backpropagation	10
2.4	Automatic Differentiation and Practical Training	11

1 Introduction to Automatic Image Analysis

Automatic image analysis starts from the central difficulty of vision, then moves through the main processing levels, the recurring goals of the field, and the practical question of how to evaluate whether a vision system is actually reliable.

1.1 Lecture Aim and the Inverse Problem

Automatic image analysis is the problem of turning measurements into explanations. A digital image is easy to store: it is a grid of values. It is much harder to understand: we want to infer objects, surfaces, motion, geometry, lighting, and meaning from those values.

Computer vision is the computational study of how useful descriptions of the visual world can be recovered from images and video. The description may be semantic, such as “there is a pedestrian in front of the car”; geometric, such as “this surface is two meters away”; or operational, such as “reject this part because it contains a defect”. This definition is deliberately broader than object recognition. A vision system may label an image, reconstruct a three-dimensional scene, align several images, estimate motion, improve an image for a human observer, or supply measurements for a larger decision-making system.

Computer vision is therefore not just image enhancement. It is inference under uncertainty. The image is evidence; the scene that produced it is hidden.

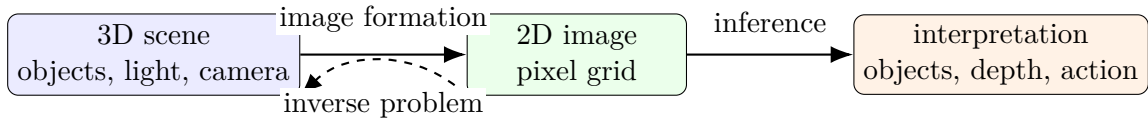


Figure 1: Computer vision tries to invert the image formation process: pixels are observed, but the scene explanation is hidden.

Human vision feels immediate, but this ease is misleading. A camera records a two-dimensional projection of a three-dimensional world. During that projection, many causes are mixed together: object shape, material, illumination, viewpoint, occlusion, sensor noise, and motion. Different scenes can produce very similar images, so there is usually no unique answer that can be read directly from the pixels.

This makes vision an inverse problem. The forward direction is comparatively clear: if we know the scene, the light, the camera, and the materials, we can model how an image is formed. The inverse direction asks us to go backward from image measurements to a plausible explanation of the world. That backward step requires assumptions, models, data, and algorithms.

A compact way to write the forward model is

$$I(x, y) = f(\Pi(X, Y, Z), L, M, C) + \varepsilon, \quad (1)$$

where $I(x, y)$ is the measured pixel value, Π is the camera projection from 3D to 2D, L describes illumination, M material and surface properties, C camera response, and ε sensor noise. Vision asks for some hidden scene state S from the image I :

$$\hat{S} = \arg \max_S p(S | I) = \arg \max_S p(I | S) p(S). \quad (2)$$

The likelihood $p(I | S)$ says which images a scene would probably produce. The prior $p(S)$ encodes what kinds of scenes are plausible before seeing the image.

The gap between the pixel array and the human interpretation is the essential problem. A person immediately infers objects, depth, material, intent, and context, while the measured image itself only contains sampled intensities or colors.

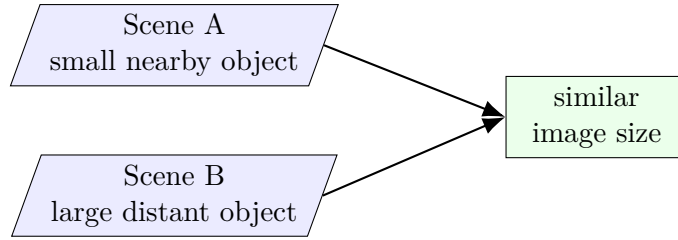


Figure 2: Ambiguity is unavoidable: different scenes can lead to similar image measurements. Priors and additional evidence help choose an explanation.

1.2 From Pixels to Interpretation

Automatic image analysis can be organized as a progression from low-level to high-level tasks.

- Low-level processing improves or transforms image measurements, for example acquisition, sampling, denoising, filtering, contrast changes, and edge extraction. Its typical mapping is image to image, or scene to image when the acquisition process is included.
- Mid-level analysis groups measurements into more meaningful structures, such as regions, contours, features, segments, motion fields, or depth estimates. It is often the bridge from image measurements to feature descriptions.
- High-level interpretation assigns semantic meaning, such as recognizing objects, estimating scene layout, detecting abnormalities, or tracking actions. Its typical mapping is from features or image evidence to object models, object detections, categories, or decisions.

These levels are useful for teaching, but real systems often mix them. A modern recognition model may learn low-level filters and high-level categories inside one network. A classical pipeline may use carefully designed features followed by statistical inference. In both cases, the goal is the same: reduce the ambiguity of image data until a useful decision or description becomes possible.

Level	Typical mapping	Typical question	Examples
Low-level	scene/image → image	What local measurements are reliable?	denoising, gradients, edges
Mid-level	image → features	Which measurements belong together?	regions, contours, segments, motion
High-level	features/image → objects	What does the image mean?	objects, scene layout, actions

Table 1: A teaching taxonomy for image-analysis tasks.

The same hierarchy can also be read as a contrast between bottom-up and top-down processing. A bottom-up system starts with the image data and builds larger hypotheses from local evidence: pixels become edges, edges become contours, contours become regions, and regions become object candidates. This is data-driven and attractive because the image itself constrains the result. Its weakness is that local evidence is often ambiguous or missing.

Top-down processing starts from a model, context, task goal, or prior expectation and asks how the image should be interpreted under that hypothesis. This is knowledge-driven: a rough object model can guide the search for parts, a scene context can make one interpretation more plausible than another, and a classifier can project learned category knowledge onto the image. Top-down processing is powerful because it uses what is already known, but it can also produce confident misinterpretations when the prior is wrong. Practical systems therefore often use a

mixed mode: bottom-up evidence proposes or constrains hypotheses, while top-down models select, refine, or reject them.

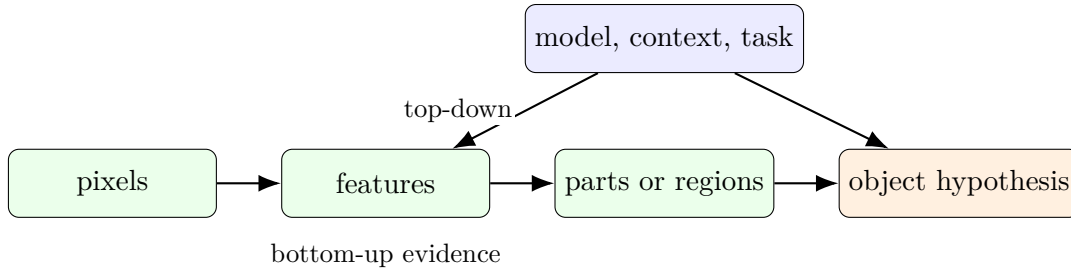


Figure 3: Bottom-up processing builds interpretations from image evidence. Top-down processing uses models and context to guide or correct the interpretation.

Computer vision can be organized around three recurring goals: recognition, reconstruction, and reorganization.

Recognition. Recognition asks what is present and where it is. Examples include classification, detection, segmentation, reading text, identifying faces, or finding defects in industrial inspection.

Reconstruction. Reconstruction asks what spatial or physical structure produced the image. Examples include estimating depth, camera motion, three-dimensional shape, or the geometry of a scene from one or more images.

Reorganization. Reorganization changes the image representation so that later analysis becomes easier or the image becomes more useful to a person. Examples include registration, stitching, stabilization, denoising, super-resolution, and computational photography.

These goals are connected. Recognition benefits from good representations; reconstruction needs reliable correspondences; reorganization often depends on understanding motion, geometry, or image formation.

Area	Main transformation	Typical examples
Digital image processing	image $\rightarrow$ image	enhancement, restoration, filtering, compression
Photogrammetry and geometry	image(s) $\rightarrow$ 3D model	camera calibration, depth, surface reconstruction, mapping
Automatic image analysis	image $\rightarrow$ object model or detection	feature extraction, segmentation, object categorization, recognition

Table 2: Automatic image analysis is closest to the recognition branch of computer vision: it studies how image evidence becomes object-level descriptions, while still using processing and geometric ideas when they support that goal.

Another useful taxonomy separates the vocabulary of the field by the kind of question being asked. Some terms describe the *input* and its formation, some describe *representations* computed from the image, some describe *tasks*, and some describe the *knowledge* used to resolve ambiguity. Keeping these groups separate helps avoid a common beginner confusion: a feature, a model, a task, and an application are not the same kind of object.

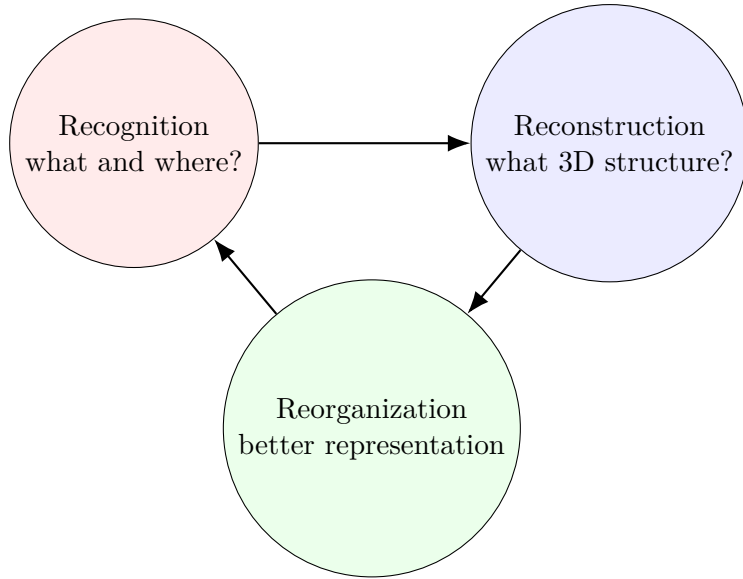


Figure 4: Recognition, reconstruction, and reorganization are not isolated topics; each goal supplies information that can support the others.

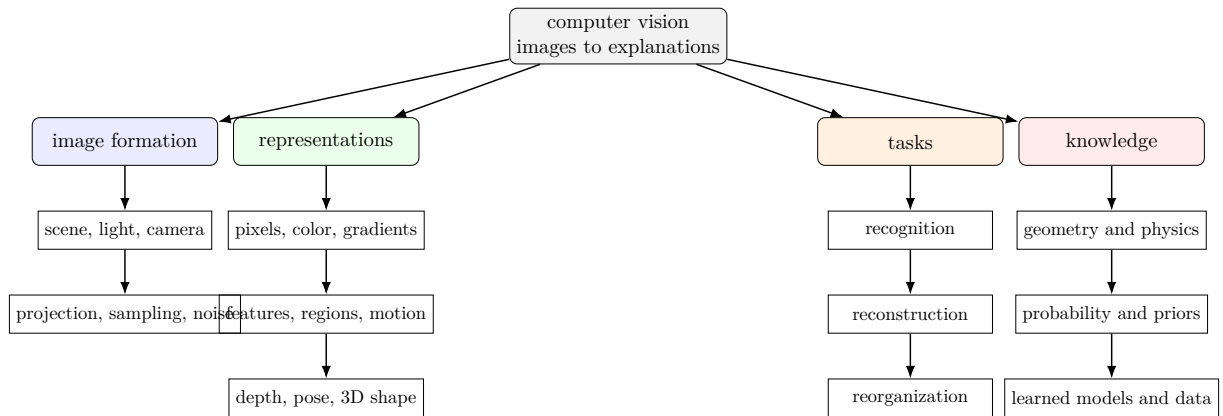


Figure 5: A compact taxonomy of introductory computer-vision terms. The field connects image formation, intermediate representations, task goals, and prior knowledge.

1.3 Data Types and Interpretation Tasks

Before listing concrete tasks, it helps to be precise about what kind of data an image is. We distinguish *structured* from *unstructured* data by whether the arrangement of the values carries meaning. A list of attributes such as (70 kg, 170 cm, 32 years) is unstructured: the entries can be permuted without changing what is described, as long as we remember which field is which. An image is structured, because the spatial layout of the pixels *is* the information. Shuffling the pixels destroys the content even though the multiset of intensity values is unchanged; sound and text are structured for the same reason. This is exactly why image analysis cannot treat a picture as an unordered bag of intensities: position and neighborhood are part of the signal, and the methods in later sections are built to exploit them.

High-level interpretation is not one task but a family of related questions, and it is worth naming them precisely because they differ in what the output looks like. *Image classification* assigns the whole image, or a given region, to one of a fixed set of categories, without saying where anything is. *Object detection* adds localization: it returns a set of objects, each with a position, usually a bounding box, together with a category, so it answers “what” and “where” at once. *Semantic segmentation* pushes localization to the pixel level, assigning a class label

to every pixel and thereby partitioning the image into semantically uniform regions, though without separating individual instances of the same class. *Object recognition* is the broader goal of identifying which objects are present; in its narrow sense it means recognizing a specific identity or instance, such as a particular face or landmark, and in its broad sense it is used as an umbrella over classification and detection. Beyond these, interpretation also includes pose estimation, which recovers the spatial configuration of an object, action recognition, which labels an activity in a video, and attribute classification, which describes properties rather than category.

Task	Output	Question it answers
Image classification	one category for the whole image or region	what is in this image?
Object detection	set of located objects, e.g. boxes plus labels	what is present and where?
Semantic segmentation	a class label for every pixel	which class does each pixel belong to?
Object recognition	identity or category of the objects present	what object is this?
Pose estimation	spatial configuration, e.g. key-points or orientation	how is the object arranged?
Action recognition	activity label for a clip	what is happening over time?

Table 3: High-level interpretation tasks differ mainly in how much localization they require, from a single image-level label up to a decision at every pixel.

These tasks share the same underlying processing chain. Image acquisition and preprocessing produce reliable low-level measurements, mid-level analysis groups them into features and segments, and the high-level stage turns those into the task-specific output above. The choice of task fixes the form of the answer, but the bottom-up evidence and top-down knowledge feeding into it are the same.

1.4 Models, Data, and Evaluation

Because images are ambiguous, vision systems need prior knowledge. Sometimes that knowledge is explicit: camera geometry, physical image formation, smooth surfaces, rigid motion, or probabilistic noise models. Sometimes it is learned from data: neural networks discover features and decision rules from many examples.

Classical computer vision often emphasizes explicit models and engineered features. Modern deep learning often emphasizes learned representations and large datasets. The practical lesson is not that one replaces the other completely. Robust systems usually combine several kinds of knowledge: scientific models for image formation, statistical models for uncertainty, engineering choices for reliability, and data-driven learning for complex patterns that are hard to specify by hand.

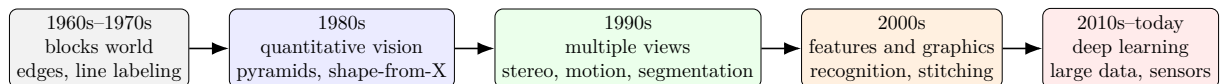


Figure 6: A brief historical timeline. The emphasis moved from hand-built geometric reasoning, through quantitative reconstruction and engineered features, toward large-scale learned representations and integrated sensing.

The timeline is not a story of one idea simply replacing another. Edge operators, camera geometry, variational estimation, probabilistic models, feature matching, and neural networks all remain

useful. What changes over time is the balance between explicit modeling, available computation, available training data, and the kinds of applications that can be deployed reliably.

Source of knowledge	What it contributes	Risk if missing
Physical model	Camera geometry, illumination, projection	Results may violate how images are formed
Statistical model	Noise, uncertainty, priors, inference	The system may overtrust ambiguous measurements
Engineering design	Robustness, speed, failure handling	The method may work only in demonstrations
Data-driven learning	Features and decision rules from examples	Complex visual variation may be underspecified

Table 4: A practical vision system usually combines several kinds of knowledge.

Computer vision is now a basic technology in many domains. It is more useful to name the application categories than to memorize a long list of products: consumer imaging and media, medicine and life sciences, industry and logistics, robotics and autonomous systems, remote sensing and mapping, document analysis, security and biometrics, and scientific measurement. The tasks differ, but the same conceptual questions recur: what image evidence is available, what assumptions make the interpretation possible, and what failure cases matter in the application context?

Application category	Typical vision output	Main reliability concern
Consumer and media	enhanced images, alignment, tracking	visual quality and robustness across devices
Medicine and life sciences	measurements, segmentations, suspected findings	uncertainty, bias, and clinical validation
Industry and logistics	defects, codes, counts, object poses	speed, repeatability, and rare failures
Robotics and autonomy	localization, obstacles, maps, actions	safety under changing environments
Remote sensing and mapping	land cover, 3D structure, change detection	scale, resolution, and registration accuracy
Documents and biometrics	text, identity cues, authenticity signals	privacy, fairness, and adversarial misuse

Table 5: Categorical application areas are easier to study than isolated product examples because they expose recurring reliability questions.

These applications also show why evaluation matters. A result that looks plausible on one example is not enough. Vision algorithms must be tested against noise, changing illumination, clutter, unusual viewpoints, occlusion, and data that differ from the training or development examples.

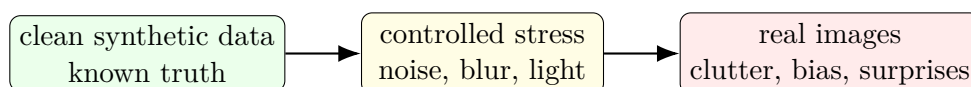


Figure 7: Evaluation should move from controlled correctness toward real-world robustness.

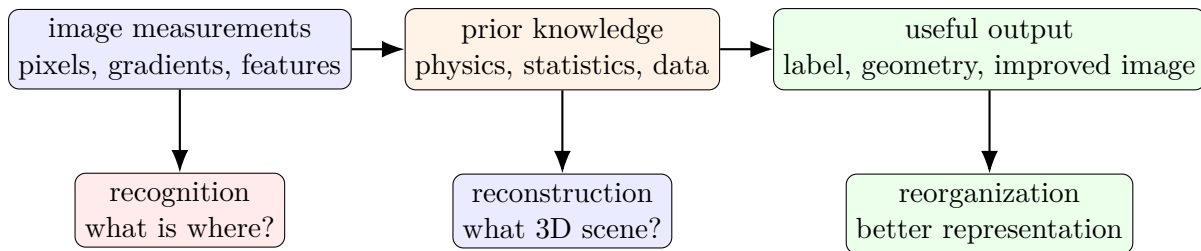


Figure 8: A compact summary of the introductory framing: measurements become useful outputs only after they are combined with assumptions, models, and data.

1.5 Topic Roadmap

Automatic image analysis is narrower than the whole field of computer vision. It uses image processing tools and needs some geometric and probabilistic reasoning, but the main line is automatic interpretation: how images are transformed into features, segments, object models, detections, categories, and evaluated decisions.

The foundations are image representation, filtering, features, transformations, and the basic language of image formation. These topics supply the bottom-up evidence used later. Segmentation, clustering, and feature extraction provide mid-level descriptions that make object-level reasoning possible. Learning-based methods, especially neural networks, show how useful representations and classifiers can be estimated from data. Object models, probabilistic estimation, and higher-level recognition tasks connect these pieces; reconstruction appears where it helps explain image formation, geometry, or the limits of recognition.

Three questions recur throughout automatic image analysis:

1. What information is actually present in the image measurements?
2. What assumptions are needed to infer the hidden scene or decision?
3. How can we test whether the algorithm works beyond the example in front of us?

Automatic image analysis starts with pixels but does not end there. Its real task is to connect images to explanations: what is in the scene, how it is arranged, how it changes, and what action or decision should follow.

2 Neural Networks and Backpropagation

Neural networks are one way to replace manually designed decision rules by learned functions. For image analysis, the input may be an image, a patch, or a feature vector, and the output may be a class score, a probability, a segmentation label, or another task-specific prediction. The central idea is simple: compose many parameterized transformations and fit their parameters from examples.

Szeliski's treatment of deep learning, especially the part on deep neural networks and backpropagation, is a useful companion for this topic. The focus here is narrower: understand the computational structure of a feed-forward network and why backpropagation is the efficient way to compute the gradients needed for training.

2.1 From Perceptrons to Multilayer Networks

A perceptron is a linear classifier. Given an input vector x , weights w , and bias b , it predicts a binary label by thresholding an affine score:

$$f(x) = \begin{cases} 1, & w^\top x + b > 0, \\ -1, & \text{otherwise.} \end{cases} \quad (3)$$

The decision boundary is the hyperplane $w^\top x + b = 0$. This makes the perceptron easy to interpret, but also limited: if the classes cannot be separated by a linear boundary, the standard perceptron cannot solve the problem. The XOR problem is the usual minimal example.

During perceptron training, an incorrectly predicted example changes the weights in the direction of the input:

$$w_i^{t+1} = w_i^t + \eta (y - f(x)) x_i, \quad (4)$$

where η is the learning rate. The rule is useful historically because it already shows the pattern of learning from prediction error, but modern neural networks need a smoother and more general gradient-based view.

A multilayer perceptron (MLP) overcomes the purely linear limitation by stacking several affine maps and nonlinear activation functions. For layer l ,

$$h^l = W^l x^l + b^l, \quad y^l = \phi(h^l), \quad x^{l+1} = y^l. \quad (5)$$

Without the nonlinearity ϕ , stacking layers would still collapse into one linear transformation. The nonlinearities make it possible to build complex decision boundaries from many simple units.

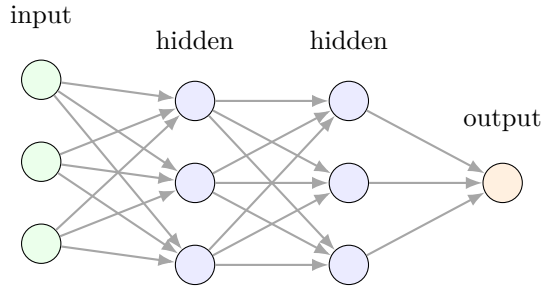


Figure 9: A feed-forward network composes affine transformations and nonlinearities. Training changes the weights on the connections.

Beyond being nonlinear, a useful transfer function is typically monotonic and smooth, so that it has a well-defined derivative everywhere; the classical logistic sigmoid and hyperbolic tangent are additionally bounded, which keeps activations in a fixed range. Common activation functions include the logistic sigmoid, hyperbolic tangent, and rectified linear unit:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z) (1 - \sigma(z)), \quad (6)$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z). \quad (7)$$

For ReLU, the derivative is 1 for positive inputs and 0 for negative inputs; at zero one chooses a convenient subgradient in implementations. This simple derivative is one reason ReLU-style activations are computationally attractive.

2.2 Losses, Optimization, and the Need for Gradients

Training starts with examples (x_i, y_i) and a network f_θ with parameters θ , meaning all weights and biases. A loss function measures how bad a prediction is. Over a dataset, the empirical

objective is

$$L(\theta) = \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i). \quad (8)$$

For regression, ℓ may be a squared error. For classification, it is often cross-entropy applied to class scores or probabilities. The exact loss depends on the task, but the training problem has the same form: find parameters that make the loss small.

Stochastic gradient descent updates parameters using a small batch $\mathcal{B}_t$ instead of the full dataset:

$$\theta_{t+1} = \theta_t - \alpha \sum_{i \in \mathcal{B}_t} \frac{\partial \ell_i(\theta_t)}{\partial \theta}, \quad (9)$$

where α is the learning rate. The practical difficulty is not writing this update down; it is computing the partial derivatives for millions of parameters quickly and without duplicating work. Backpropagation solves exactly that problem.

Component	Role in the network	Training question
Weights and biases	Store the adjustable model parameters	How should each parameter change to reduce the loss?
Activations	Introduce nonlinear transformations between layers	Which units are active and how do they transmit gradients?
Loss function	Converts predictions and targets into one scalar	What does “better” mean for this task?
Optimizer	Turns gradients into parameter updates	How large and in which direction should the next step be?

Table 6: Training a neural network requires both a model architecture and a procedure for turning prediction errors into parameter updates.

2.3 Backpropagation

Backpropagation is an efficient application of the chain rule to a computational graph. The forward pass computes the network output and stores intermediate quantities. The backward pass starts from the derivative of the loss with respect to the output and propagates sensitivity information back through the same operations in reverse order.

It is helpful to name the quantities at one layer before writing the gradient. Let h_k be the activation vector that enters layer k . The trainable parameters of this layer are a weight matrix Ω_k and a bias vector β_k . The affine part of the layer first forms the pre-activation vector

$$f_k = \beta_k + \Omega_k h_k, \quad (10)$$

and a nonlinearity would then produce the next activation, for example $h_{k+1} = \phi(f_k)$. With this notation, the gradient with respect to the weight matrix has the structure

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial f_k} h_k^\top. \quad (11)$$

This formula captures a useful intuition from the lecture: the effect of a weight depends both on the upstream error signal and on the activation feeding into that weight. If the previous activation is doubled, the same small weight change has twice the local effect.

For ReLU hidden units, the backward recurrence includes the pointwise derivative of the activation:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = \mathbb{I}[f_{k-1} > 0] \odot \left(\Omega_k^\top \frac{\partial \ell_i}{\partial f_k} \right), \quad (12)$$

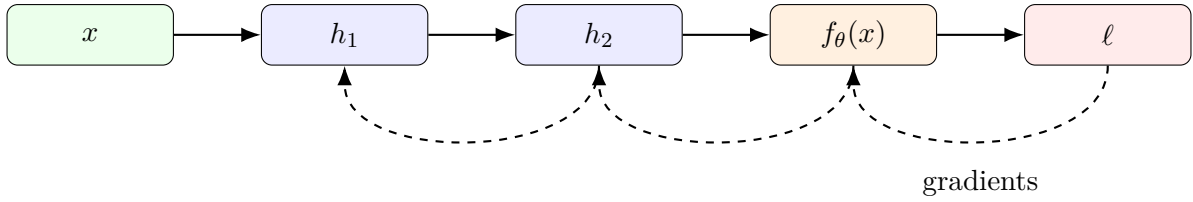


Figure 10: The forward pass evaluates the network from left to right. The backward pass propagates derivatives from the loss back through the stored intermediate quantities.

where $\odot$ denotes elementwise multiplication. The indicator gates the gradient: inactive ReLU units do not pass local gradient information backward.

The important point is not to memorize every indexed expression, but to know what must be stored and what is reused. The activations from the forward pass are needed during the backward pass. This makes backpropagation efficient, but also memory-hungry for large networks.

2.4 Automatic Differentiation and Practical Training

Modern deep-learning libraries implement backpropagation through automatic differentiation. A model is represented as an acyclic graph of operations. Each operation knows how to compute its output in the forward direction and how to propagate derivatives in the reverse direction. The user specifies the model, the loss, and the optimizer; the framework records the graph and applies the chain rule.

A typical training loop has the following structure:

1. initialize the network parameters;
2. choose a loss function and optimizer;
3. load a batch of examples;
4. run the forward pass and compute the loss;
5. run the backward pass to compute gradients;
6. update the parameters and repeat.

This is the conceptual core behind PyTorch-style code: zero old gradients, evaluate the model, call backward on the loss, and let the optimizer step.

Several practical issues remain outside the pure gradient formula. Poor initialization can make training slow or unstable. Too large a learning rate can overshoot; too small a learning rate can make progress painfully slow. Deep models may fit the training data while generalizing poorly. In the usual bias–variance language, underfitting corresponds to high bias, while overfitting corresponds to high variance. Regularization, more data, data augmentation, dropout, weight decay, batch normalization, and early stopping are common ways to improve generalization.

Symptom	Meaning	Typical response
High training error	The model or optimization setup is too weak	increase capacity, train longer, adjust learning rate
Low training error but high validation error	The model overfits the training set	use regularization, more data, or early stopping
Unstable loss	Updates are too aggressive or scales are poor	lower learning rate, improve initialization, normalize inputs

Table 7: Most neural-network training failures can be diagnosed by comparing training behavior with validation behavior.